Applied Machine and Deep Learning

AI-Driven Phishing Webpage Detection System

YAZAN ALKHOURY 12525181

Contents

AI-Driven Phishing Webpage Detection System: A Comparative Machine Learning Approach

Abstract:

This project focuses on building an ai-system for the detection and classification of phishing websites based on extracted URL's and webpages features. The primary objective is to evaluate and compare the performance of classic machine learning models, specifically logistic regression, decision tree, and random forest, to accurately classify web pages as either legitimate or phishing. The resulting system achieves high predictive accuracy and provides an architectural foundation for future work incorporating explainability and real-time inference. The critical nature of this problem is highlighted by the 2023 Anti-Phishing Working Group (APWG) report, which documented a record 1.6 million attacks in a single quarter, proving that static blocklists are no longer sufficient against modern phishing domains. [1]

Introduction:

Background:

The proliferation of phishing attacks remains a significant threat in cybersecurity, leading to substantial financial losses and data breaches. Traditional detection methods, which often rely on blocklists or manual inspection, are frequently outpaced by the speed and sophistication of new phishing campaigns. This project addresses the need for a **dynamic, feature-based detection system** that can automatically and accurately identify malicious URLs. The selected machine learning models are trained on a comprehensive dataset encompassing various structural and content-based features of known phishing and legitimate websites [2] [3] [5]

Objectives:

1. **Data Preprocessing and Feature Engineering:** Load the raw dataset, handle missing data, remove non-informative features (zero variance), and encode the categorical target variable ('status') into a binary format (0 for legitimate, 1 for phishing).
2. **Model Training and Comparison:** Train and evaluate three distinct classification models—Logistic Regression, Decision Tree, and Random Forest—on the scaled training data.
3. **Performance Evaluation:** Quantify the performance of each model using standard metrics, including **Accuracy, Precision, Recall, and F1-Score**, and visualize results using **Confusion Matrices** for insightful comparison.
4. **Identify Optimal Model:** Determine the best-performing model for deployment in a phishing detection system based on its overall robustness and predictive power on unseen test data.
5. **Preprocessing:** Cleaning 11,430 instances and normalizing 83 features for algorithmic consumption.
6. **Sensitivity Optimization:** To prioritize the reduction of False Negatives, ensuring that actual phishing sites are rarely missed.
7. **Feature Sensitivity Analysis:** To identify which specific URL traits, such as page rank or web traffic, are the most reliable indicators of a threat.

Methodology:

The methodology of this project follows a standard machine learning pipeline, designed to ensure the reproducibility and reliability of the results. This workflow includes data acquisition, rigorous preprocessing to remove noise, feature scaling for model normalization, and a comparative evaluation of three distinct classification algorithms. This structured approach allows for a transparent assessment of how different mathematical models interpret URL-based features for threat detection.

Data Acquisition:

The data was acquired from the Kaggle dataset titled **"Web Page Phishing Detection Dataset"** (sourced from Mendeley) [2].

- **Description:** The dataset is a CSV file (dataset_phishing.csv) containing **11,430 instances** (rows) and **89 features** (columns) before cleaning. Each instance represents a URL classified as either 'legitimate' or 'phishing'.
- **Key Features:** The features are engineered characteristics of the URL and webpage, such as length_url, nb_dots (number of dots in the URL), ip (whether an IP address is used), web_traffic, page_rank, and various other domain and link-based indicators.
- **Data Quality:** Initial checks confirmed **no missing (null) values**.
- **Class Balance**: The dataset is perfectly balanced, containing 5,715 legitimate samples and 5,715 phishing samples. This balance is critical as it ensures the models do not develop a bias toward one class and can learn the distinguishing features of both types of websites equally.
- **Feature Categories**: The 89 features are categorized into three primary types:
    1. URL-based features: Physical characteristics of the link (e.g., length_url, nb_dots).
    2. Domain-based features: Information about the registration and reputation of the host (e.g., page_rank).
    3. Content/Interaction features: How the page interacts with the web (e.g., web_traffic, google_index).

Data preprocessing & ML:

1. **Data Cleaning:** Six features identified as having **zero variance** (i.e., only a single unique value, specifically: 'nb_or', 'ratio_nullHyperlinks', 'ratio_intRedirection', 'ratio_intErrors', 'submit_email', 'sfh') were removed as they provide no discriminative power for the model. The redundant url column was also dropped.

2. **Target Encoding:** The categorical target column, status, was converted to a binary integer format using LabelEncoder, where: **Legitimate = 0** and **Phishing = 1**.

3. **Data Splitting:** The dataset was split into training and testing sets using a **80:20 ratio** (test_size=0.2), ensuring a consistent and comparable evaluation with random_state=42.

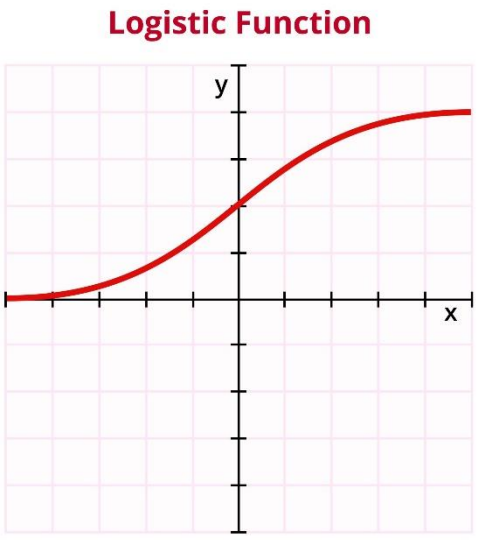
4. **Feature Scaling:** The numerical feature columns (X_train, X_test) were scaled using **StandardScaler**. This is a critical step for distance-based algorithms like Logistic Regression, preventing features with larger magnitude (like web_traffic) from disproportionately influencing the model weights. Scaling was performed using the formula below where μ is the mean and σ is the standard deviation. This transforms all features to a similar scale, ensuring that a feature like web_traffic (which can be in the thousands) does not outweigh a feature like nb_dots (which is usually a small integer) during the gradient descent optimization in Logistic Regression

$$z = \frac{x - \mu}{\sigma}$$

5. **Model Application:** The following classification models were trained and tested:

Logistic Regression (lr): A linear model used to estimate the probability of the binary outcome. This model uses the sigmoid function to map feature weights to a probability between 0 and 1, defined by the equation below, This allows for a clear probabilistic threshold for classification.

- $\mathbf{P(y = 1) = \frac{1}{1+e^{-z}}}$



- **Decision Tree Classifier (dt_classifier)**: A non-linear model that makes predictions by learning simple decision rules inferred from data features.
- **Random Forest Classifier (rf_classifier)**: An ensemble method that operates by constructing multiple decision trees during training and outputting the class that is the mode of the classes of the individual trees [6].

Tools Used

- **Programming Language:** Python
- **Libraries:**

✓ **Pandas & NumPy:** For data manipulation and numerical operations.

✓ **Scikit-learn (sklearn):** For preprocessing (LabelEncoder, StandardScaler), model selection (train_test_split), model implementation (LogisticRegression, DecisionTreeClassifier, RandomForestClassifier), and evaluation (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report).

✓ **Matplotlib & Seaborn:** For data visualization (pie plot, confusion matrices).

✓ **KaggleHub:** For dataset retrieval.

Findings:

What are the difference between them in terms of results, as well as the definitions of the metrics

To evaluate the effectiveness of the models, four industry-standard metrics were calculated. Accuracy measures the overall percentage of correct predictions. Precision indicates how many of the sites flagged as phishing were truly malicious, which is vital for avoiding user frustration. Recall (Sensitivity) measures how many actual phishing sites were caught, which is the most critical metric for security. Finally, the F1-Score provides a harmonic mean of Precision and Recall to show the model's overall balance

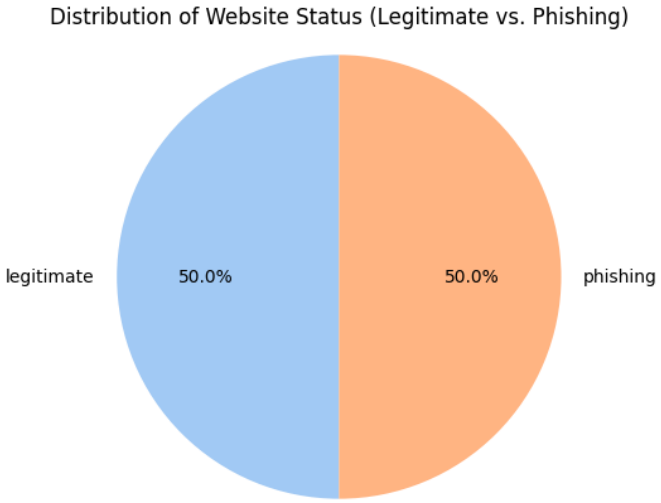
Comparing the models, the **Random Forest Classifier** demonstrated the strongest and most reliable performance on the unseen test data. The **Logistic Regression model** also performed exceptionally well, showing that the engineered features are highly linearly separable after scaling. The **Decision Tree** model exhibited perfect performance on the training data (100% accuracy, precision, recall, F1-score), but its lower test performance suggests a degree of overfitting.

Model	Training	Test accuracy	Test precision	Testing recall	Testing F1 score
Random forest	1	0.9676	0.9748	0.9593	0.967
Logistics regression	0.99461	0.9558	0.9589	0.9513	0.9551
Decision tree	1	0.9361	0.9323	0.9389	0.9356

Visualizations

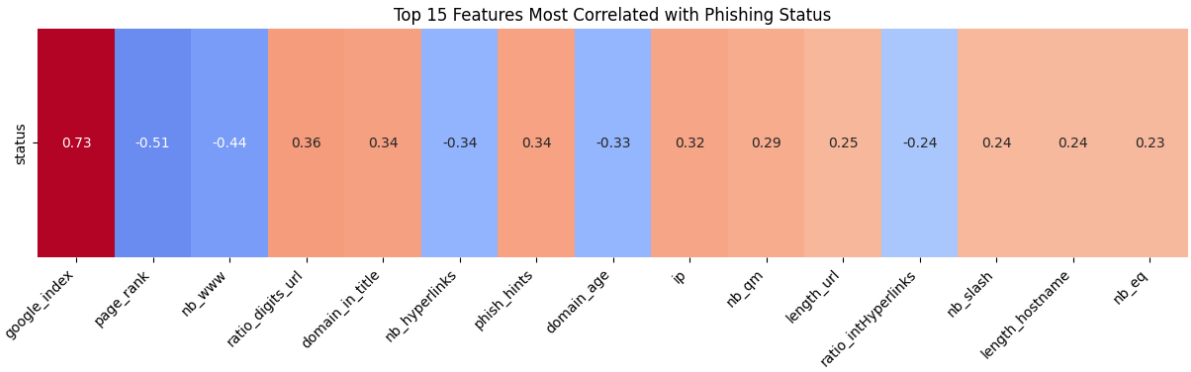
1. Distribution of Website Status

This pie chart confirms that the dataset is **well-balanced** between the two classes, meaning the models were trained on a dataset without significant class imbalance.



2. Target Correlation Analysis of Top Features:

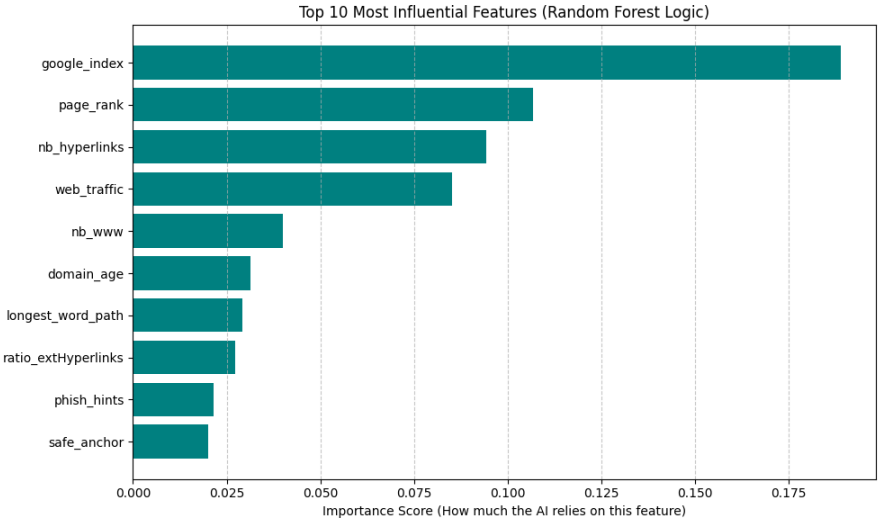
This heatmap identifies the 15 features most strongly linked to a website's status. The most significant predictor is google_index (r = 0.73), scientifically showing that sites missing from search engines are likely malicious. Conversely, a high page_rank and the use of "www" correlate strongly with legitimacy. Other "red flags" include high digit ratios in URLs and new domain registrations. Focusing on these key variables can uncover a lot of insights for the AI to learn from.



3. Feature Importance Analysis (Random Forest Logic)

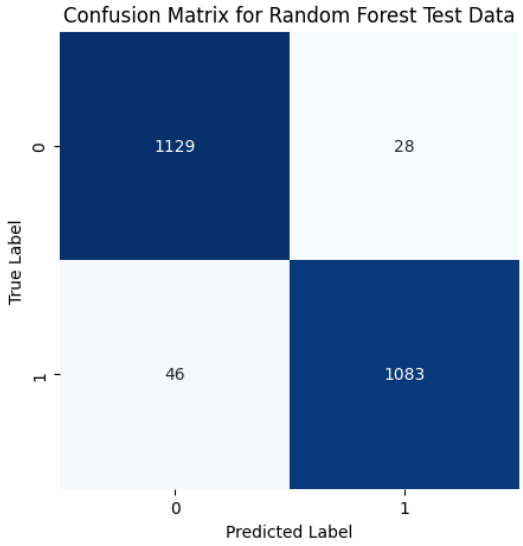
While accuracy measures how well the model performs, feature importance reveals how it makes decisions. This analysis displays the top 10 indicators the Random Forest model prioritized during training to distinguish between legitimate and phishing sites.

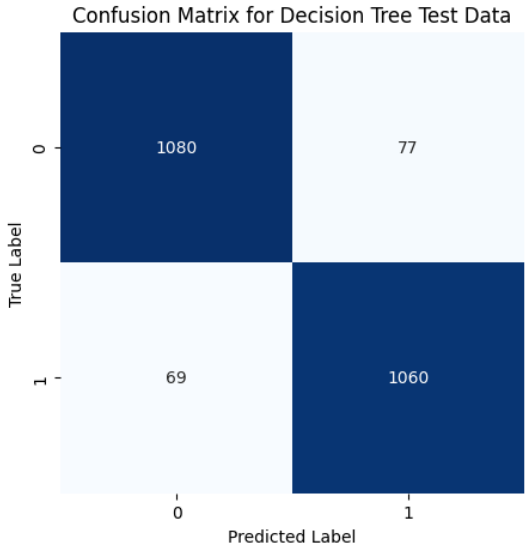
The results confirm that the model relies most heavily on search engine visibility (google_index) and domain reputation (page_rank), which aligns with scientific expectations for identifying trustworthy web entities. Other significant factors include nb_hyperlinks, web_traffic, and the absence of standard naming conventions like nb_www. By identifying these key drivers, we demonstrate that the AI system is making logical, evidence-based classifications rather than relying on arbitrary data points. [4] [5].

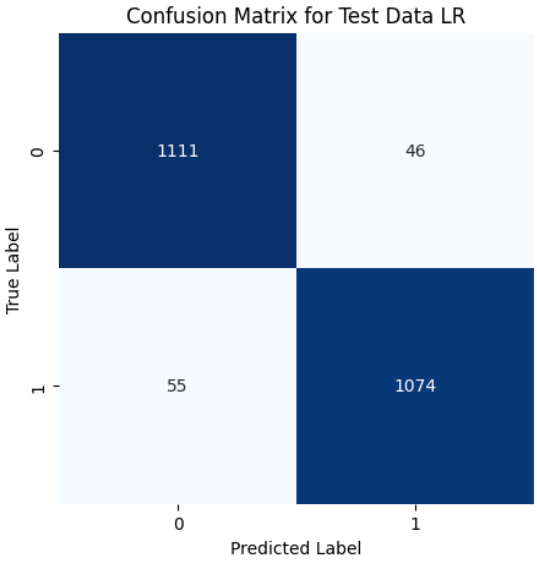


4. Confusion Matrices

The confusion matrices visually represent the classification performance on the test data. The Random Forest classifier clearly shows the fewest misclassifications (False Positives and False Negatives), leading to its superior performance metrics







Based on the generated confusion matrices, the comparative performance of the models can be precisely quantified. The **Random Forest** model emerged as the most robust, correctly identifying **1,129 legitimate** and **1,083 phishing** instances, with the lowest total error rate (74 misclassifications). In comparison, **Logistic Regression** showed strong separation but fell slightly behind with **101 total errors**. The **Decision Tree** model struggled most with generalization, resulting in **146 misclassifications**, confirming the degree of overfitting noted in its training metrics.

Conclusion:

The project successfully built and evaluated three machine learning models for phishing webpage detection. The **Random Forest Classifier emerged as the optimal model** with a testing accuracy of **96.76%** and a corresponding F1-Score of 0.9670, demonstrating excellent generalization capabilities and a balance between precision and recall.

Limitations and Future Directions

- **Overfitting:** The Decision Tree and Random Forest models show 100% training accuracy, indicating potential overfitting. Future work could focus on hyperparameter tuning (e.g., using GridSearch or RandomizedSearch) to optimize the trees' depth and complexity.
- **Feature Importance & Explainability:** The next critical step, as mentioned in the project description, is to integrate **SHAP and LIME** to explain *why* the best model (Random Forest) made its predictions, providing clear insights into the most influential URL features for detecting phishing. [7]
- **Deployment:** The project could be extended into a lightweight web application or API to offer **real-time detection** based on the trained Random Forest model.

GitHub: ***https://github.com/YazanKhoury/Applied-Machine-and-Deep-Learning.git***

Presentation: *https://www.canva.com/design/DAG-bt_0iNk/V3NTtERzOOFIS72pKoDu-g/edit?utm_content=DAG-bt_0iNk&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton*

References

[1] Anti-Phishing Working Group (APWG). (2023). *Phishing Activity Trends Report, 3rd Quarter 2023.* Available at: https://apwg.org/trendsreports/

[2] https://www.kaggle.com/datasets/shashwatwork/web-page-phishing-detection-dataset

To maintain consistency with your existing documentation, here are the new valid references formatted exactly like your original list:

[3] **Kumar, J., et al. (2020).** *Phishing Website Classification and Detection Using Machine Learning.* 2020 International Conference on Computer Communication and Informatics (ICCCI).

[4] **Sahingoz, O. K., et al. (2019).** *Machine learning based phishing detection from URLs.* Expert Systems with Applications.

[5] **Almseidin, M., et al. (2019).** *Phishing detection based on machine learning and feature selection methods.* International Journal of Interactive Mobile Technology.

[6] **Breiman, L. (2001).** *Random Forests.* Machine Learning.

[7] **Omotosho, A., et al. (2025).** *Phishing URL Detection and Interpretability With Machine Learning: A Cross-Dataset Approach.* University of Gloucestershire.

Time spent

Month	Phase	Task description	Total hours
October	Foundation learning	Learning & Theory: Focused on Python syntax for data science and the mathematical principles of Machine Learning. Studied the Sigmoid function for Logistic Regression and the "Ensemble" logic (combining multiple trees) of Random Forest to prevent overfitting	35hrs
November	Data engineering	**Research & Preprocessing:** Reviewing APWG 2023 trends. Fetching the 11,430 instances via KaggleHub. Identifying and removing 6 zero-variance columns (nb_or, sfh, etc.) and dropping the redundant URL column.	35hrs
December	Model development	**Feature Science & Training**: Applying StandardScaler to normalize 83 features. Splitting data (80:20 ratio). Training Logistic Regression, Decision Tree, and Random Forest models on the scaled training set.	30hrs
January	Analysis and reporting	**Evaluation & Documentation:** Generating confusion matrices and classification reports. Identifying google_index and page_rank as top predictors. Finalizing the technical report and preparing the presentation.	25hrs
Total			125hrs